

In [2]: `pip install wordcloud`

```
Collecting wordcloud
  Using cached wordcloud-1.9.6-cp39-cp39-win_amd64.whl (306 kB)
Requirement already satisfied: numpy>=1.19.3 in c:\users\shrey\anaconda3\lib\site-packages (from wordcloud) (1.24.4)
Requirement already satisfied: matplotlib in c:\users\shrey\anaconda3\lib\site-packages (from wordcloud) (3.5.2)
Requirement already satisfied: pillow in c:\users\shrey\anaconda3\lib\site-packages (from wordcloud) (11.3.0)
Requirement already satisfied: cyclor>=0.10 in c:\users\shrey\anaconda3\lib\site-packages (from matplotlib->wordcloud) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\shrey\anaconda3\lib\site-packages (from matplotlib->wordcloud) (4.25.0)
Requirement already satisfied: packaging>=20.0 in c:\users\shrey\anaconda3\lib\site-packages (from matplotlib->wordcloud) (21.3)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\shrey\anaconda3\lib\site-packages (from matplotlib->wordcloud) (1.4.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\shrey\anaconda3\lib\site-packages (from matplotlib->wordcloud) (2.8.2)
Requirement already satisfied: pyparsing>=2.2.1 in c:\users\shrey\anaconda3\lib\site-packages (from matplotlib->wordcloud) (3.0.9)
Requirement already satisfied: six>=1.5 in c:\users\shrey\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib->wordcloud) (1.16.0)
Installing collected packages: wordcloud
Successfully installed wordcloud-1.9.6
Note: you may need to restart the kernel to use updated packages.
```

```
WARNING: Ignoring invalid distribution -andas (c:\users\shrey\anaconda3\lib\site-packages)
WARNING: Ignoring invalid distribution -andas (c:\users\shrey\anaconda3\lib\site-packages)
WARNING: Ignoring invalid distribution -andas (c:\users\shrey\anaconda3\lib\site-packages)
WARNING: Ignoring invalid distribution -andas (c:\users\shrey\anaconda3\lib\site-packages)
WARNING: Ignoring invalid distribution -andas (c:\users\shrey\anaconda3\lib\site-packages)
WARNING: Ignoring invalid distribution -andas (c:\users\shrey\anaconda3\lib\site-packages)
WARNING: Ignoring invalid distribution -andas (c:\users\shrey\anaconda3\lib\site-packages)
```

```
In [3]: # =====
# Twitter Sentiment Analysis
# Cell 1 - Import Libraries
# =====

# Data Handling
import pandas as pd
import numpy as np

# Text Processing
import re
import string

# NLP
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

# Visualizations
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
```

```

# Machine Learning (used later)
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

# Word Cloud
from wordcloud import WordCloud

# Save Model
import joblib

# Ignore warnings
import warnings
warnings.filterwarnings("ignore")

# Download NLTK resources
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')

print("Libraries Loaded Successfully")

```

```

[nltk_data] Downloading package punkt to
[nltk_data]   C:\Users\shrey\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]   C:\Users\shrey\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\stopwords.zip.
[nltk_data] Downloading package wordnet to
[nltk_data]   C:\Users\shrey\AppData\Roaming\nltk_data...
[nltk_data] Downloading package omw-1.4 to
[nltk_data]   C:\Users\shrey\AppData\Roaming\nltk_data...
Libraries Loaded Successfully

```

```

In [4]: # =====
# Cell 2 - Load Twitter Dataset
# =====

# Training Dataset
train = pd.read_csv(
    "twitter_training.csv",
    header=None,
    names=["ID", "Brand", "Sentiment", "Tweet"]
)

```

```

)

# Validation Dataset
validation = pd.read_csv(
    "twitter_validation.csv",
    header=None,
    names=["ID", "Brand", "Sentiment", "Tweet"]
)

# Merge datasets
df = pd.concat([train, validation], ignore_index=True)

print("Dataset Loaded Successfully\n")

print("Shape :", df.shape)

print("\nColumns")

print(df.columns)

print("\nFirst Five Rows")

display(df.head())

print("\nMissing Values")

display(df.isnull().sum())

```

Dataset Loaded Successfully

Shape : (75682, 4)

Columns

Index(['ID', 'Brand', 'Sentiment', 'Tweet'], dtype='object')

First Five Rows

	ID	Brand	Sentiment	Tweet
0	2401	Borderlands	Positive	im getting on borderlands and i will murder yo...
1	2401	Borderlands	Positive	I am coming to the borders and I will kill you...
2	2401	Borderlands	Positive	im getting on borderlands and i will kill you ...
3	2401	Borderlands	Positive	im coming on borderlands and i will murder you...
4	2401	Borderlands	Positive	im getting on borderlands 2 and i will murder ...

Missing Values

```
ID          0
Brand       0
Sentiment   0
Tweet      686
dtype: int64
```

```
In [5]: # =====
# Cell 3 - Data Cleaning
# =====

# Remove duplicate rows
df.drop_duplicates(inplace=True)

# Remove rows with missing tweets
df.dropna(subset=["Tweet"], inplace=True)

# Reset index
df.reset_index(drop=True, inplace=True)

print("Dataset Shape After Cleaning :", df.shape)

# -----
# Stopwords and Lemmatizer
# -----

stop_words = set(stopwords.words("english"))

lemmatizer = WordNetLemmatizer()

# -----
# Text Cleaning Function
# -----

def clean_text(text):

    text = str(text).lower()

    # Remove URLs
    text = re.sub(r'http\S+', '', text)

    # Remove Mentions
    text = re.sub(r'@\w+', '', text)

    # Remove Hashtags
    text = re.sub(r'#', '', text)

    # Remove Numbers
    text = re.sub(r'\d+', '', text)
```

```

# Remove Punctuation
text = text.translate(
    str.maketrans('', '', string.punctuation)
)

# Remove Extra Spaces
text = re.sub(r'\s+', ' ', text).strip()

# Tokenization
words = word_tokenize(text)

cleaned_words = []

for word in words:

    if word not in stop_words:

        word = lemmatizer.lemmatize(word)

        cleaned_words.append(word)

    return " ".join(cleaned_words)

# -----
# Apply Cleaning
# -----

print("Cleaning Tweets...")

df["Clean_Tweet"] = df["Tweet"].apply(clean_text)

print("Completed")

# Tweet Length Feature
df["Tweet_Length"] = df["Clean_Tweet"].apply(len)

print("\nSample Cleaned Tweets")

display(
    df[
        ["Tweet", "Clean_Tweet", "Sentiment"]
    ].head(10)
)

# Save Clean Dataset
df.to_csv("twitter_clean.csv", index=False)

print("\nClean dataset saved as twitter_clean.csv")

```

Dataset Shape After Cleaning : (72139, 4)

Cleaning Tweets...

Completed

Sample Cleaned Tweets

	Tweet	Clean_Tweet	Sentiment
0	im getting on borderlands and i will murder yo...	im getting borderland murder	Positive
1	I am coming to the borders and I will kill you...	coming border kill	Positive
2	im getting on borderlands and i will kill you ...	im getting borderland kill	Positive
3	im coming on borderlands and i will murder you...	im coming borderland murder	Positive
4	im getting on borderlands 2 and i will murder ...	im getting borderland 2 murder	Positive
5	im getting into borderlands and i can murder y...	im getting borderland murder	Positive
6	So I spent a few hours making something for fu...	spent hour making something fun dont know huge...	Positive
7	So I spent a couple of hours doing something f...	spent couple hour something fun dont know im h...	Positive
8	So I spent a few hours doing something for fun...	spent hour something fun dont know im huge bor...	Positive
9	So I spent a few hours making something for fu...	spent hour making something fun dont know huge...	Positive

Clean dataset saved as twitter_clean.csv

```
In [6]: # =====
# Cell 4 - Exploratory Data Analysis (EDA)
# =====

import json

print("=" * 60)
print("Exploratory Data Analysis")
print("=" * 60)

# -----
# Dataset Information
# -----
print("\nDataset Shape:")
print(df.shape)

print("\nDataset Info:")
print(df.info())

print("\nSummary Statistics:")
display(df.describe(include='all'))

# -----
# Sentiment Distribution
# -----
```

```

print("\nSentiment Distribution:")
sentiment_counts = df["Sentiment"].value_counts()

display(sentiment_counts)

# -----
# Brand Distribution
# -----
print("\nTop 10 Brands:")

brand_counts = df["Brand"].value_counts().head(10)

display(brand_counts)

# -----
# Tweet Length Statistics
# -----
print("\nTweet Length Statistics:")

display(df["Tweet_Length"].describe())

# -----
# Save Statistics
# -----
stats = {
    "Total Tweets": int(len(df)),
    "Unique Brands": int(df["Brand"].nunique()),
    "Sentiment Classes": int(df["Sentiment"].nunique()),
    "Average Tweet Length": round(df["Tweet_Length"].mean(), 2),
    "Maximum Length": int(df["Tweet_Length"].max()),
    "Minimum Length": int(df["Tweet_Length"].min()),
    "Sentiment Distribution": sentiment_counts.to_dict()
}

with open("stats.json", "w") as f:
    json.dump(stats, f, indent=4)

print("\nstats.json saved successfully.")

```

=====

Exploratory Data Analysis

=====

Dataset Shape:
(72139, 6)

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 72139 entries, 0 to 72138
Data columns (total 6 columns):
Column Non-Null Count Dtype
--- -
0 ID 72139 non-null int64
1 Brand 72139 non-null object
2 Sentiment 72139 non-null object
3 Tweet 72139 non-null object
4 Clean_Tweet 72139 non-null object
5 Tweet_Length 72139 non-null int64
dtypes: int64(2), object(4)
memory usage: 3.3+ MB
None

Summary Statistics:

	ID	Brand	Sentiment	Tweet	Clean_Tweet	Tweet_Length
count	72139.000000	72139	72139	72139	72139	72139.000000
unique	NaN	32	4	69974	63269	NaN
top	NaN	TomClancysRainbowSix	Negative	At the same time, despite the fact that there ...		NaN
freq	NaN	2349	21790	172	1510	NaN
mean	6435.469593	NaN	NaN	NaN	NaN	75.403693
std	3743.598918	NaN	NaN	NaN	NaN	54.615670
min	1.000000	NaN	NaN	NaN	NaN	0.000000
25%	3195.000000	NaN	NaN	NaN	NaN	33.000000
50%	6432.000000	NaN	NaN	NaN	NaN	64.000000
75%	9607.000000	NaN	NaN	NaN	NaN	106.000000
max	13200.000000	NaN	NaN	NaN	NaN	813.000000

Sentiment Distribution:

```

Sentiment
Negative      21790
Positive      19846
Neutral       17879
Irrelevant    12624
Name: count, dtype: int64
Top 10 Brands:
Brand
TomClancysRainbowSix    2349
CallOfDuty              2333
Verizon                 2330
MaddenNFL              2326
Microsoft              2317
WorldOfCraft           2317
LeagueOfLegends        2314
TomClancysGhostRecon   2305
ApexLegends            2305
NBA2K                  2304
Name: count, dtype: int64
Tweet Length Statistics:
count    72139.000000
mean      75.403693
std       54.615670
min        0.000000
25%       33.000000
50%       64.000000
75%      106.000000
max       813.000000
Name: Tweet_Length, dtype: float64
stats.json saved successfully.

```

```

In [7]: # =====
# Cell 5 - Data Visualization
# =====

from collections import Counter

# -----
# Sentiment Distribution
# -----
plt.figure(figsize=(8,5))

sns.countplot(
    data=df,
    x="Sentiment",
    palette="viridis"
)

plt.title("Sentiment Distribution")
plt.xlabel("Sentiment")
plt.ylabel("Number of Tweets")

```

```

plt.xticks(rotation=15)

plt.show()

# -----
# Top Brands
# -----
plt.figure(figsize=(12,6))

sns.barplot(
    x=brand_counts.values,
    y=brand_counts.index,
    palette="mako"
)

plt.title("Top 10 Brands")
plt.xlabel("Tweets")
plt.ylabel("Brand")

plt.show()

# -----
# Tweet Length Distribution
# -----
plt.figure(figsize=(10,5))

sns.histplot(
    df["Tweet_Length"],
    bins=40,
    kde=True,
    color="steelblue"
)

plt.title("Tweet Length Distribution")
plt.xlabel("Tweet Length")

plt.show()

# -----
# Top 20 Words
# -----
all_words = " ".join(df["Clean_Tweet"]).split()

word_freq = Counter(all_words)

common_words = word_freq.most_common(20)

words = [i[0] for i in common_words]

counts = [i[1] for i in common_words]

```

```

plt.figure(figsize=(12,6))

sns.barplot(
    x=counts,
    y=words,
    palette="rocket"
)

plt.title("Top 20 Most Frequent Words")

plt.show()

# -----
# Overall Word Cloud
# -----
text = " ".join(df["Clean_Tweet"])

wordcloud = WordCloud(
    width=1200,
    height=600,
    background_color="white",
    colormap="viridis"
).generate(text)

plt.figure(figsize=(15,7))

plt.imshow(wordcloud)

plt.axis("off")

plt.title("Overall Word Cloud")

plt.show()

# -----
# Positive Word Cloud
# -----
positive_df = df[
    df["Sentiment"].str.lower()=="positive"
]

text = " ".join(positive_df["Clean_Tweet"])

wordcloud = WordCloud(
    width=1200,
    height=600,
    background_color="white",
    colormap="Greens"
).generate(text)

plt.figure(figsize=(15,7))

```

```

plt.imshow(wordcloud)

plt.axis("off")

plt.title("Positive Tweets")

plt.show()

# -----
# Negative Word Cloud
# -----
negative_df = df[
    df["Sentiment"].str.lower()=="negative"
]

text = " ".join(negative_df["Clean_Tweet"])

wordcloud = WordCloud(
    width=1200,
    height=600,
    background_color="black",
    colormap="Reds"
).generate(text)

plt.figure(figsize=(15,7))

plt.imshow(wordcloud)

plt.axis("off")

plt.title("Negative Tweets")

plt.show()

# -----
# Brand-wise Sentiment
# -----
brand_sentiment = pd.crosstab(
    df["Brand"],
    df["Sentiment"]
)

top_brands = (
    df["Brand"]
    .value_counts()
    .head(10)
    .index
)

brand_sentiment = brand_sentiment.loc[top_brands]

```

```
brand_sentiment.plot(
    kind="bar",
    figsize=(14,6)
)

plt.title("Brand-wise Sentiment")

plt.ylabel("Tweets")

plt.xticks(rotation=45)

plt.show()

# -----
# Correlation Heatmap
# -----
temp = pd.DataFrame()

temp["Tweet_Length"] = df["Tweet_Length"]

temp["Sentiment_Code"] = (
    df["Sentiment"]
    .astype("category")
    .cat.codes
)

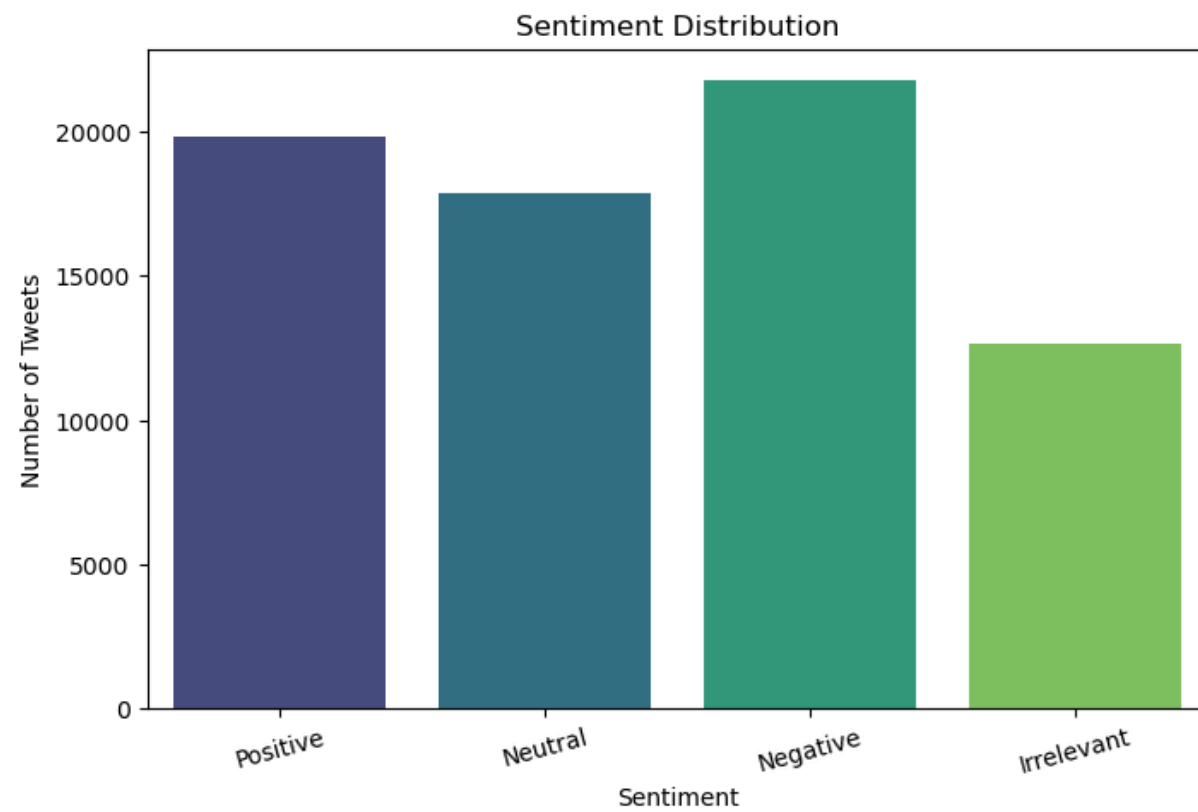
plt.figure(figsize=(6,5))

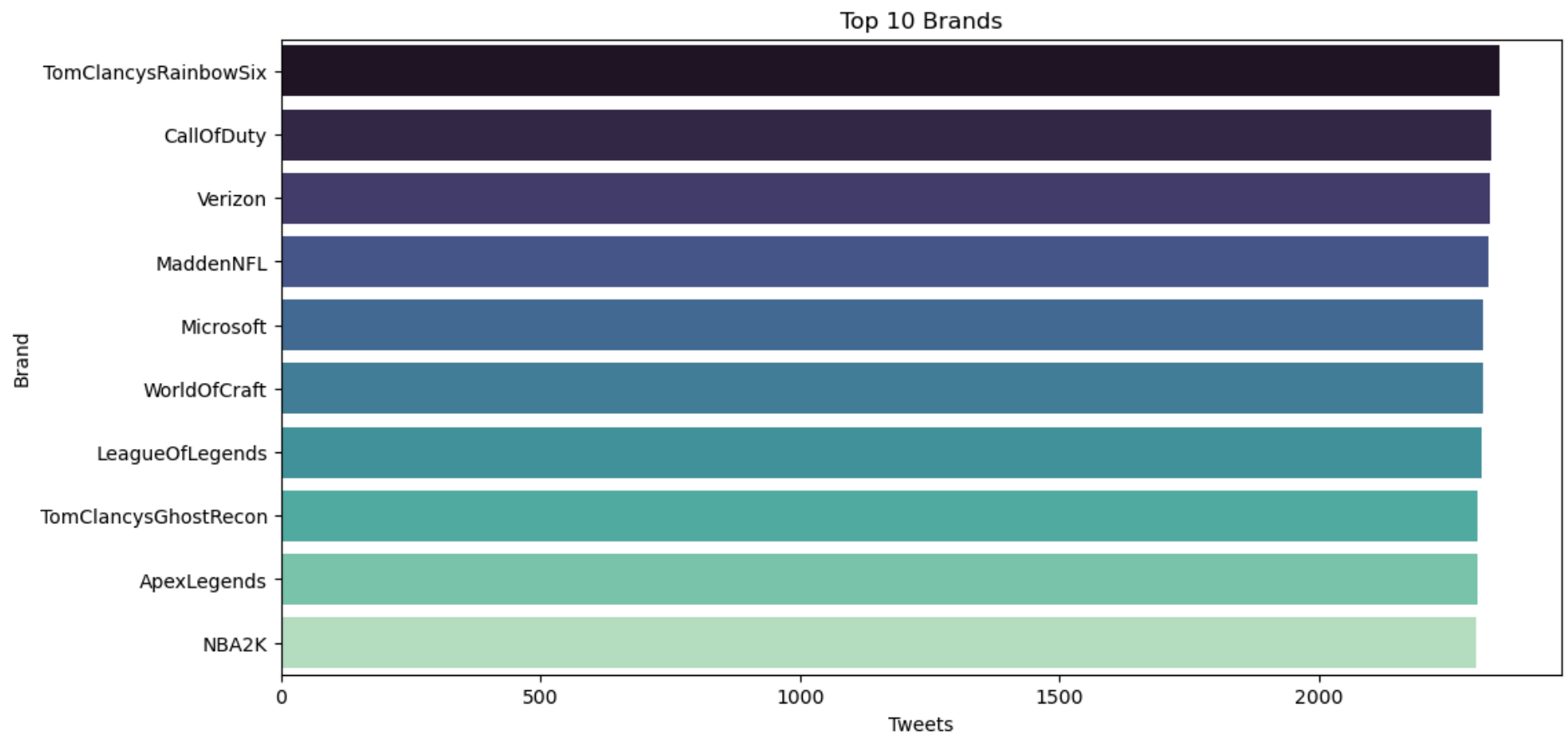
sns.heatmap(
    temp.corr(),
    annot=True,
    cmap="coolwarm"
)

plt.title("Correlation Heatmap")

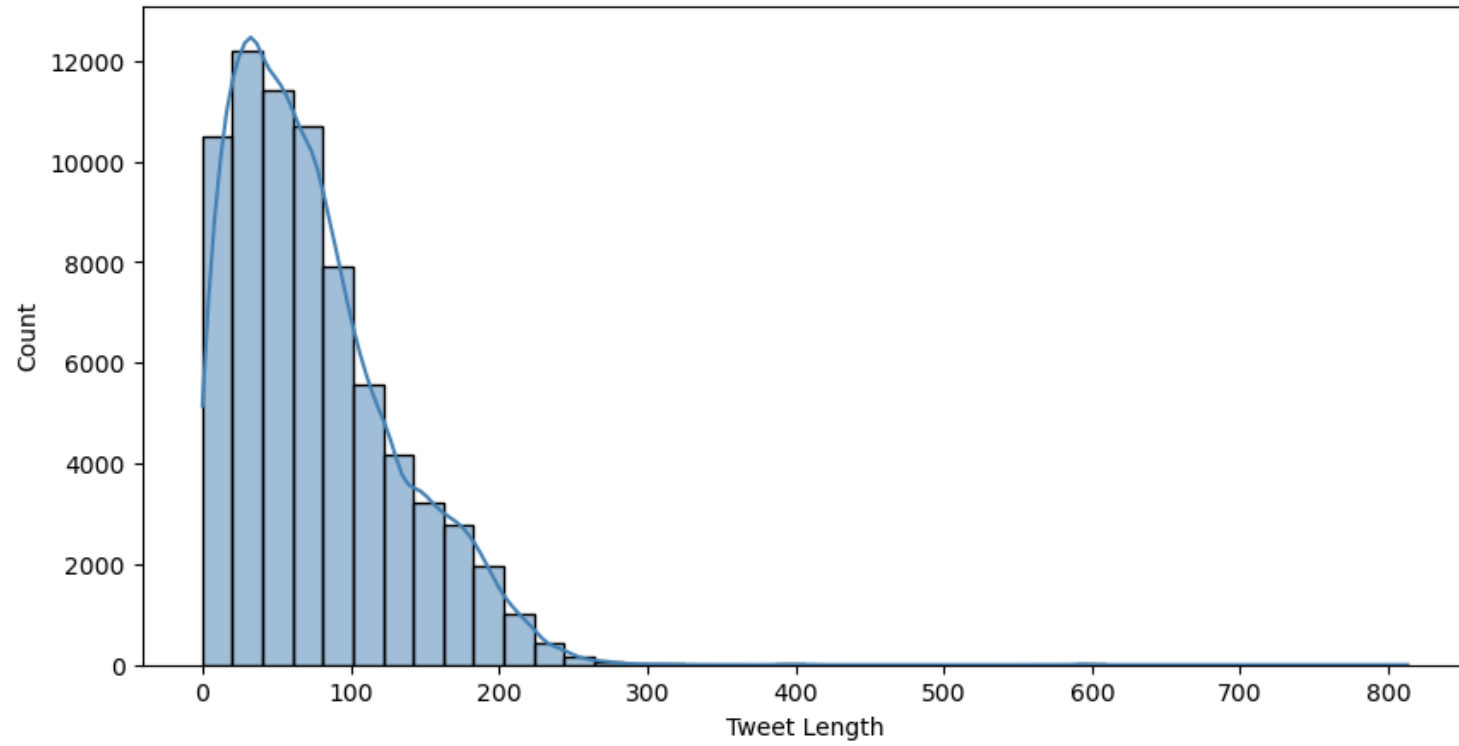
plt.show()

print("\nEDA Completed Successfully!")
```

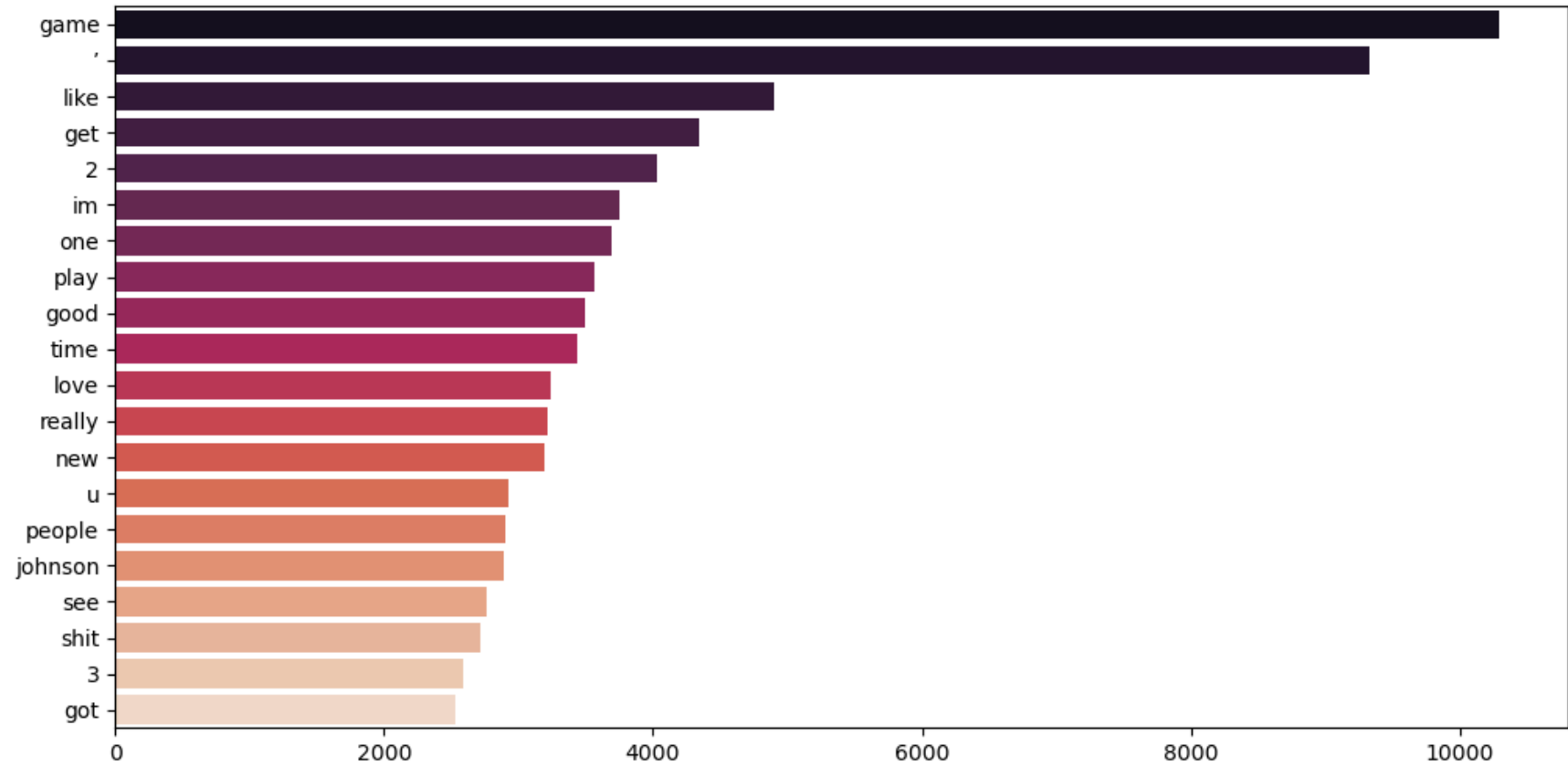




Tweet Length Distribution



Top 20 Most Frequent Words

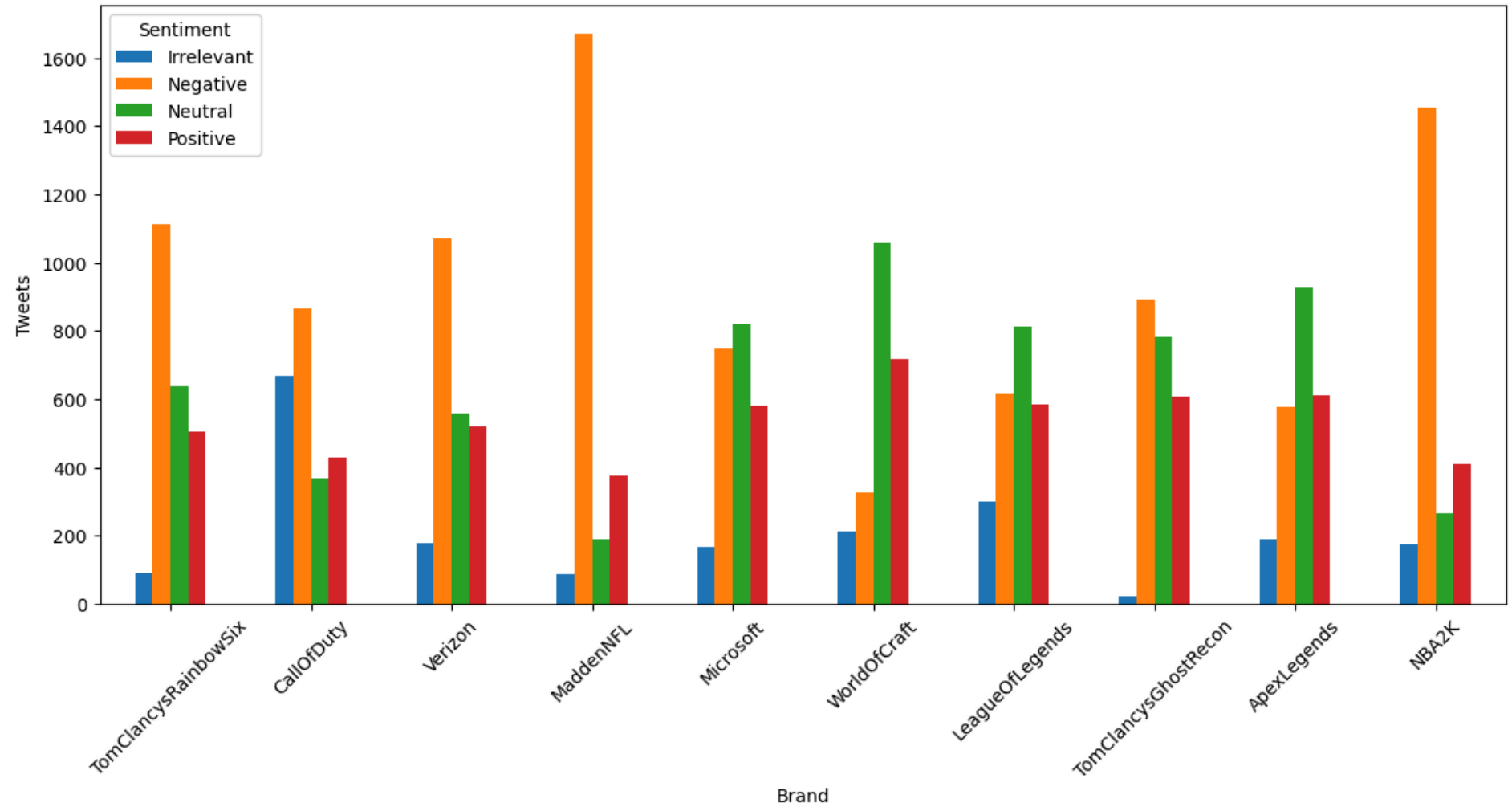


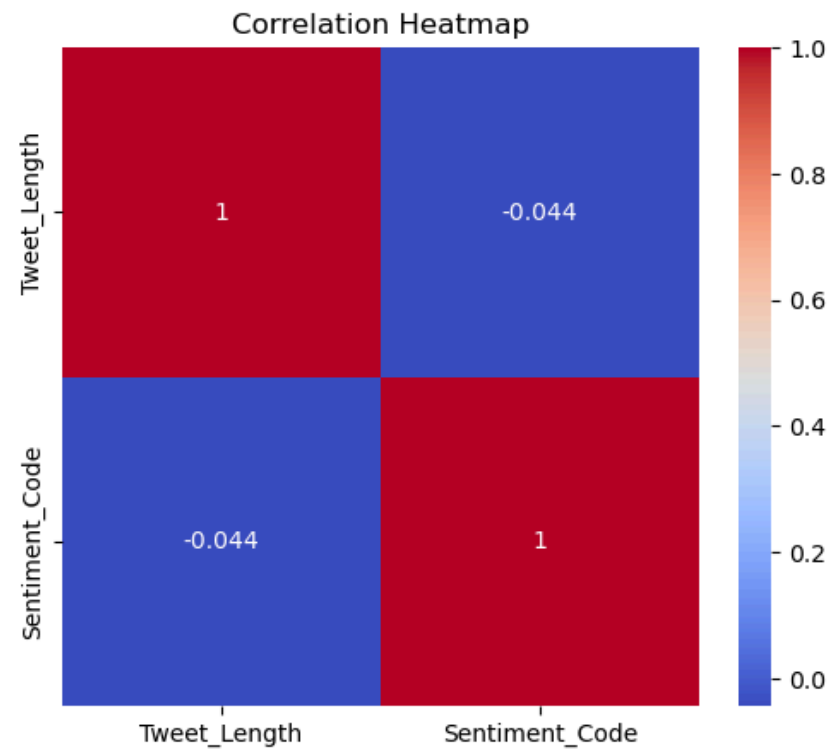
[illegible]

Negative Tweets



Brand-wise Sentiment





EDA Completed Successfully!

```
In [8]: # =====
# Cell 6 - TF-IDF & Model Training
# =====

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC

# -----
# Features & Target
# -----
X = df["Clean_Tweet"]
y = df["Sentiment"]

# -----
# Train-Test Split
# -----
X_train, X_test, y_train, y_test = train_test_split(
```

```

X,
y,
test_size=0.2,
random_state=42,
stratify=y
)

print("Training Samples :", len(X_train))
print("Testing Samples  :", len(X_test))

# -----
# TF-IDF Vectorization
# -----
vectorizer = TfidfVectorizer(
    max_features=5000,
    ngram_range=(1,2),
    stop_words="english"
)

X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

print("\nTF-IDF Shape")

print("Training :", X_train_tfidf.shape)
print("Testing  :", X_test_tfidf.shape)

# -----
# Initialize Models
# -----
models = {

    "Logistic Regression":
        LogisticRegression(max_iter=1000),

    "Naive Bayes":
        MultinomialNB(),

    "Linear SVM":
        LinearSVC()

}

print("\nModels Initialized Successfully")

```

Training Samples : 57711
Testing Samples : 14428

TF-IDF Shape
Training : (57711, 5000)
Testing : (14428, 5000)

Models Initialized Successfully

```
In [9]: # =====  
# Cell 7 - Train & Evaluate Models  
# =====  
  
from sklearn.metrics import (  
    accuracy_score,  
    precision_score,  
    recall_score,  
    f1_score,  
    classification_report,  
    confusion_matrix  
)  
  
import seaborn as sns  
import matplotlib.pyplot as plt  
import pandas as pd  
import joblib  
  
results = []  
  
best_accuracy = 0  
best_model = None  
best_model_name = ""  
  
# -----  
# Train Each Model  
# -----  
  
for name, model in models.items():  
  
    print("*60)  
    print(name)  
    print("*60)  
  
    model.fit(X_train_tfidf, y_train)  
  
    predictions = model.predict(X_test_tfidf)  
  
    accuracy = accuracy_score(y_test, predictions)  
  
    precision = precision_score(  
        y_test,
```

```

        predictions,
        average="weighted"
    )

recall = recall_score(
    y_test,
    predictions,
    average="weighted"
)

f1 = f1_score(
    y_test,
    predictions,
    average="weighted"
)

print("\nAccuracy :", round(accuracy,4))
print("Precision:", round(precision,4))
print("Recall   :", round(recall,4))
print("F1 Score :", round(f1,4))

print("\nClassification Report\n")
print(classification_report(y_test, predictions))

cm = confusion_matrix(y_test, predictions)

plt.figure(figsize=(7,5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=model.classes_,
    yticklabels=model.classes_
)

plt.title(f"{name} Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()

results.append({
    "Model":name,
    "Accuracy":accuracy,
    "Precision":precision,

```

```

        "Recall":recall,

        "F1 Score":f1

    })

    if accuracy > best_accuracy:

        best_accuracy = accuracy
        best_model = model
        best_model_name = name

# -----
# Model Comparison
# -----

results_df = pd.DataFrame(results)

print("\nModel Comparison")

display(results_df)

# -----
# Plot Comparison
# -----

results_df.set_index("Model")[[
    "Accuracy",
    "Precision",
    "Recall",
    "F1 Score"
]].plot(
    kind="bar",
    figsize=(10,6)
)

plt.title("Model Performance Comparison")

plt.ylabel("Score")

plt.ylim(0,1)

plt.grid(axis="y")

plt.show()

# -----
# Save Best Model
# -----

joblib.dump(best_model, "best_model.pkl")

```

```

joblib.dump(vectorizer, "tfidf_vectorizer.pkl")

print("\nBest Model :", best_model_name)
print("Best Accuracy :", round(best_accuracy,4))

print("\nSaved Files")
print("best_model.pkl")
print("tfidf_vectorizer.pkl")

# -----
# Save Results
# -----

results_df.to_json(
    "ml_results.json",
    orient="records",
    indent=4
)

print("ml_results.json saved successfully.")

```

=====

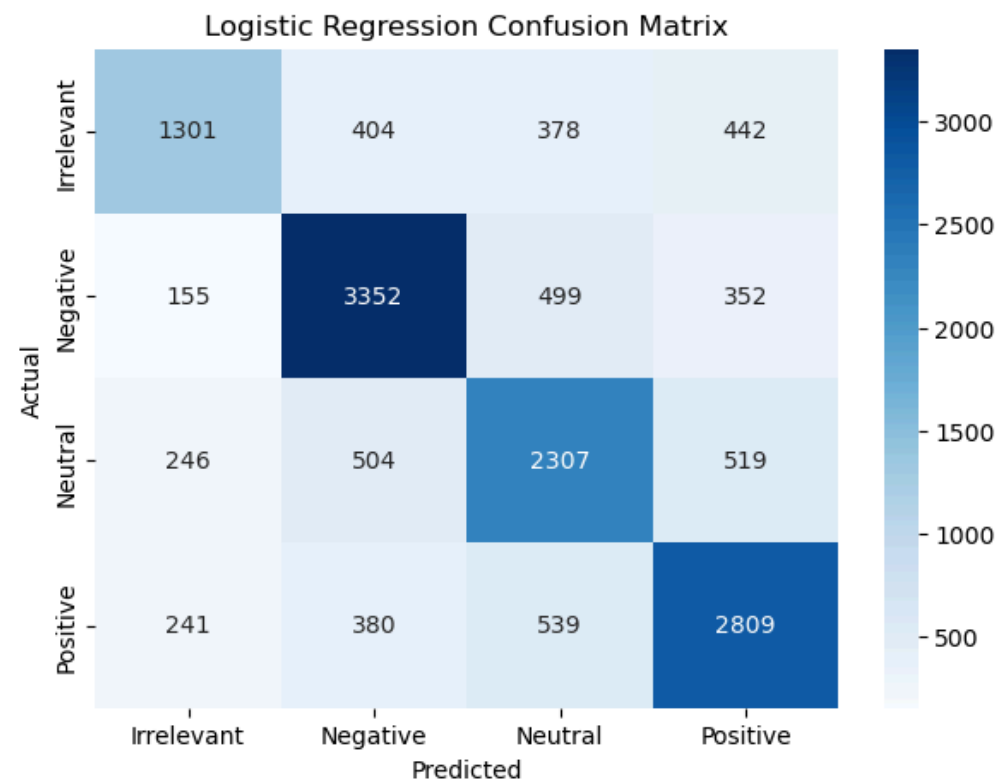
Logistic Regression

=====

Accuracy : 0.6771
Precision: 0.6764
Recall : 0.6771
F1 Score : 0.6746

Classification Report

	precision	recall	f1-score	support
Irrelevant	0.67	0.52	0.58	2525
Negative	0.72	0.77	0.75	4358
Neutral	0.62	0.65	0.63	3576
Positive	0.68	0.71	0.69	3969
accuracy			0.68	14428
macro avg	0.67	0.66	0.66	14428
weighted avg	0.68	0.68	0.67	14428

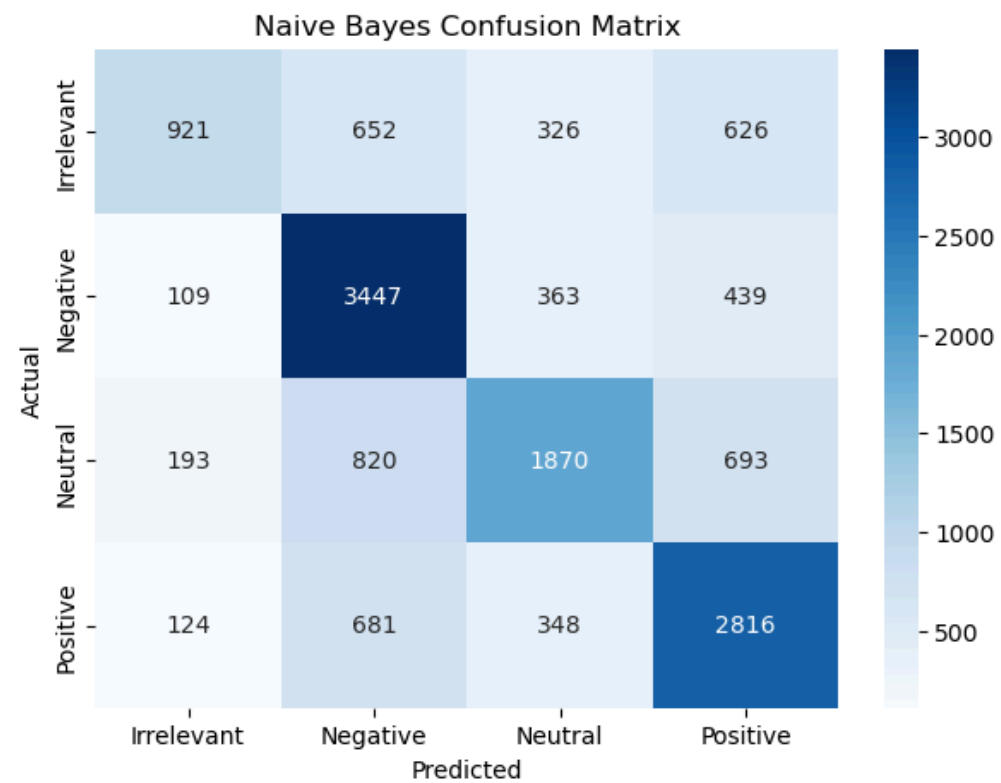


=====
Naive Bayes
=====

Accuracy : 0.6275
Precision: 0.6344
Recall : 0.6275
F1 Score : 0.6167

Classification Report

	precision	recall	f1-score	support
Irrelevant	0.68	0.36	0.48	2525
Negative	0.62	0.79	0.69	4358
Neutral	0.64	0.52	0.58	3576
Positive	0.62	0.71	0.66	3969
accuracy			0.63	14428
macro avg	0.64	0.60	0.60	14428
weighted avg	0.63	0.63	0.62	14428

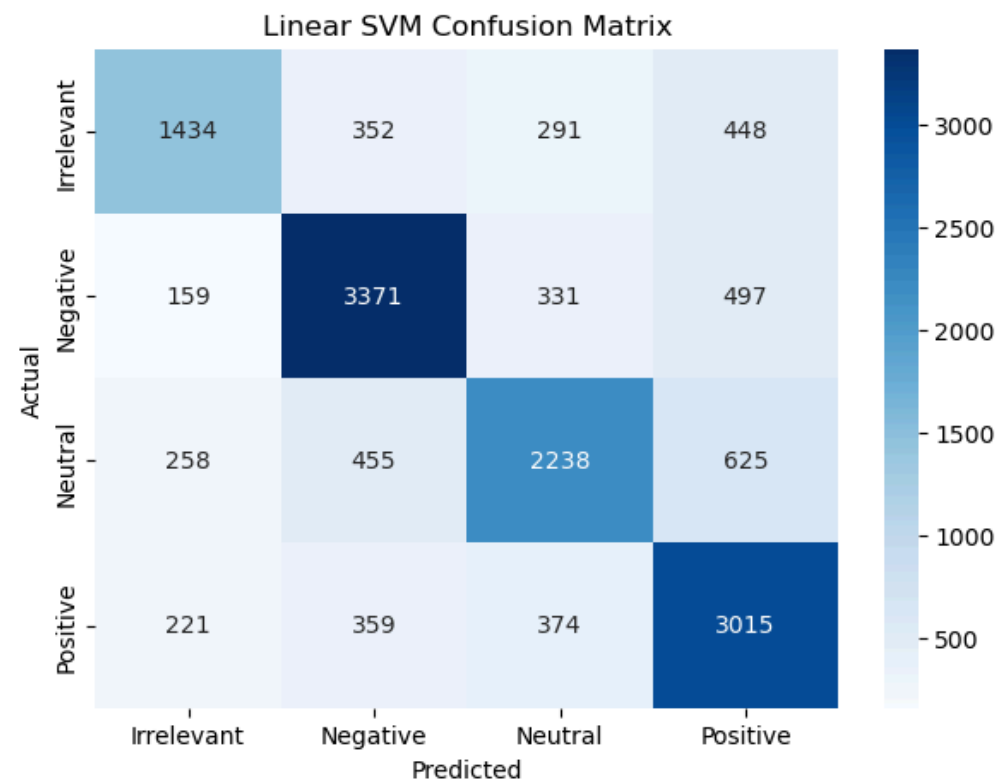


=====
Linear SVM
=====

Accuracy : 0.6971
Precision: 0.698
Recall : 0.6971
F1 Score : 0.695

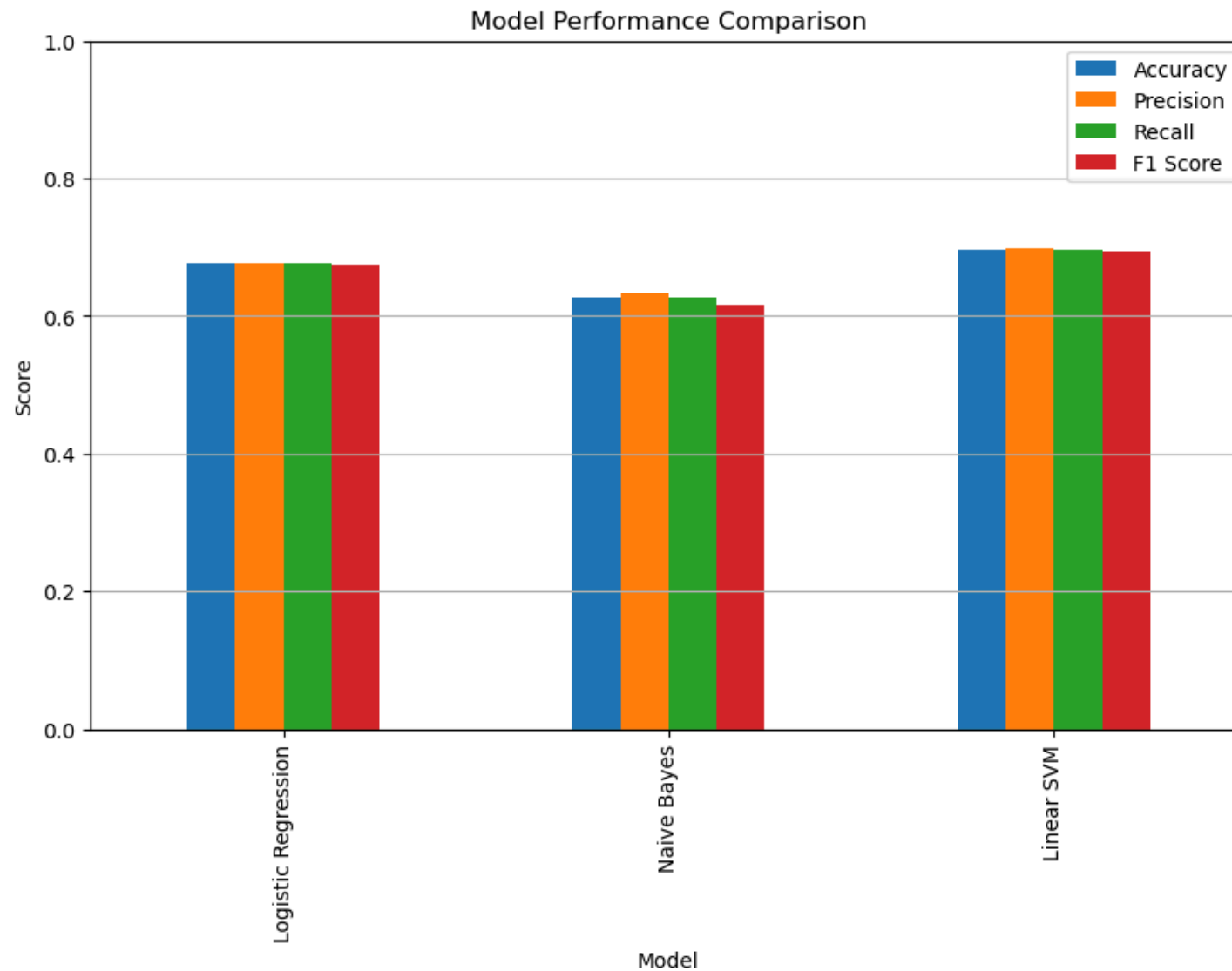
Classification Report

	precision	recall	f1-score	support
Irrelevant	0.69	0.57	0.62	2525
Negative	0.74	0.77	0.76	4358
Neutral	0.69	0.63	0.66	3576
Positive	0.66	0.76	0.70	3969
accuracy			0.70	14428
macro avg	0.70	0.68	0.69	14428
weighted avg	0.70	0.70	0.69	14428



Model Comparison

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.677086	0.676436	0.677086	0.674649
1	Naive Bayes	0.627530	0.634380	0.627530	0.616705
2	Linear SVM	0.697117	0.697957	0.697117	0.694951



Best Model : Linear SVM
Best Accuracy : 0.6971

Saved Files
best_model.pkl
tfidf_vectorizer.pkl
ml_results.json saved successfully.

```
In [10]: # =====  
# Cell 8 - Interactive Plotly Dashboard
```

```

# =====

import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# -----
# KPI Values
# -----

total_tweets = len(df)

total_brands = df["Brand"].nunique()

positive = len(df[df["Sentiment"].str.lower()=="positive"])

negative = len(df[df["Sentiment"].str.lower()=="negative"])

neutral = len(df[df["Sentiment"].str.lower()=="neutral"])

best_model_accuracy = results_df["Accuracy"].max()

best_model_name = results_df.loc[
    results_df["Accuracy"].idxmax(),
    "Model"
]

# -----
# Dashboard Layout
# -----

fig = make_subplots(
    rows=3,
    cols=2,
    specs=[
        [{"type": "indicator"}, {"type": "indicator"}],
        [{"type": "xy"}, {"type": "xy"}],
        [{"type": "domain"}, {"type": "table"}]
    ],
    subplot_titles=(
        "Total Tweets",
        "Best Model Accuracy",
        "Sentiment Distribution",
        "Top 10 Brands",
        "Sentiment Share",
        "Model Performance"
    )
)

# -----
# KPI Card 1

```

```

# -----
fig.add_trace(
    go.Indicator(
        mode="number",
        value=total_tweets,
        title={"text": "Total Tweets"}
    ),
    row=1,
    col=1
)

# -----
# KPI Card 2
# -----

fig.add_trace(
    go.Indicator(
        mode="gauge+number",
        value=best_model_accuracy*100,
        title={"text": "Best Accuracy (%)"},
        gauge={
            "axis": {"range": [0,100]},
            "bar": {"color": "green"}
        }
    ),
    row=1,
    col=2
)

# -----
# Sentiment Distribution
# -----

sentiment_counts = (
    df["Sentiment"]
    .value_counts()
    .reset_index()
)

sentiment_counts.columns = [
    "Sentiment",
    "Count"
]

fig.add_trace(
    go.Bar(
        x=sentiment_counts["Sentiment"],
        y=sentiment_counts["Count"],
        marker_color="royalblue",
        name="Sentiments"
    )
)

```

```

    ),
    row=2,
    col=1
)

# -----
# Top Brands
# -----

top_brands = (
    df["Brand"]
    .value_counts()
    .head(10)
    .reset_index()
)

top_brands.columns=[
    "Brand",
    "Tweets"
]

fig.add_trace(
    go.Bar(
        x=top_brands["Brand"],
        y=top_brands["Tweets"],
        marker_color="orange",
        name="Brands"
    ),
    row=2,
    col=2
)

# -----
# Pie Chart
# -----

fig.add_trace(
    go.Pie(
        labels=sentiment_counts["Sentiment"],
        values=sentiment_counts["Count"],
        hole=0.45
    ),
    row=3,
    col=1
)

# -----
# Model Comparison Table
# -----

fig.add_trace(

```

```

go.Table(
    header=dict(
        values=list(results_df.columns),
        fill_color="lightblue",
        align="center"
    ),
    cells=dict(
        values=[
            results_df[col]
            for col in results_df.columns
        ],
        align="center"
    )
),
row=3,
col=2
)

# -----
# Layout
# -----

fig.update_layout(

    title="Twitter Sentiment Analysis Dashboard",

    template="plotly_white",

    height=1100,

    width=1400,

    showlegend=False

)

fig.show()

# Save Dashboard

fig.write_html("twitter_sentiment_dashboard.html")

print("Dashboard Saved Successfully")

```

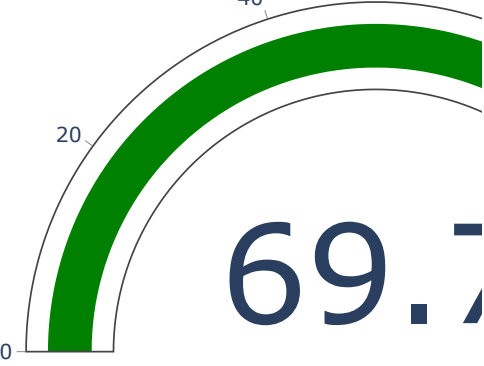
Twitter Sentiment Analysis Dashboard

Total Tweets

Total Tweets

72.1k

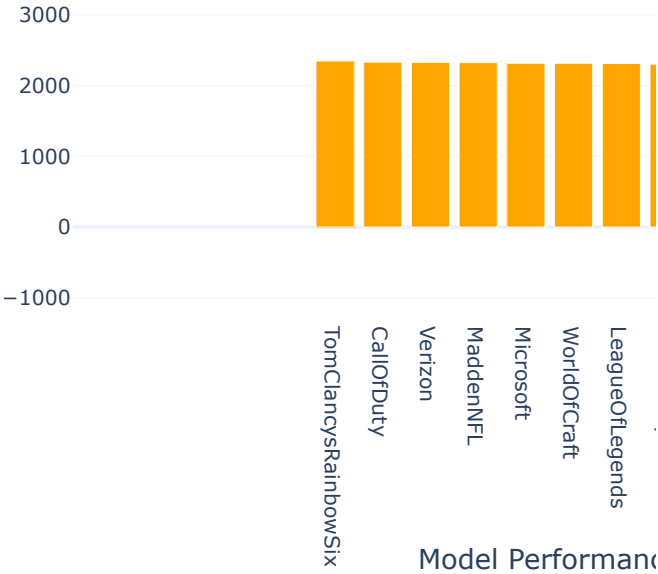
Best Accuracy (Best Model Accuracy)



Sentiment Distribution



Top 10 Brands

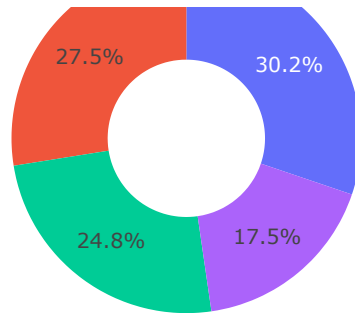


Sentiment Share



Model Performance

Model	Accuracy	Precision
-------	----------	-----------



Logistic Regression	.6770862212364846	764362519822246
Naive Bayes	.6275298031605216	343798652736356
Linear SVM	.6971167174937616	79567532224936

Dashboard Saved Successfully

```
In [11]: # =====
# Cell 9 - Predict New Tweets
# =====

import joblib

# Load Saved Model
model = joblib.load("best_model.pkl")

vectorizer = joblib.load("tfidf_vectorizer.pkl")

# -----
# Prediction Function
# -----

def predict_sentiment(text):

    cleaned = clean_text(text)

    vector = vectorizer.transform([cleaned])

    prediction = model.predict(vector)[0]

    print("*60)

    print("Original Tweet")

    print(text)

    print()
```

```
print("Cleaned Tweet")

print(cleaned)

print()

print("Predicted Sentiment")

print(prediction)

print("="*60)

# -----
# Example Predictions
# -----

predict_sentiment(
    "This game is absolutely amazing! I love playing it."
)

predict_sentiment(
    "Worst update ever. Totally disappointed."
)

predict_sentiment(
    "The graphics are okay but nothing special."
)
```

```
=====
Original Tweet
This game is absolutely amazing! I love playing it.

Cleaned Tweet
game absolutely amazing love playing

Predicted Sentiment
Positive
=====
=====
Original Tweet
Worst update ever. Totally disappointed.

Cleaned Tweet
worst update ever totally disappointed

Predicted Sentiment
Negative
=====
=====
Original Tweet
The graphics are okay but nothing special.

Cleaned Tweet
graphic okay nothing special

Predicted Sentiment
Positive
=====
```

In []: